Assignment 5: Public Key Crypto Report
CSC 3201 Intro to Computer Security
Sean LeBlanc-Grappendorf
saleblan@calpoly.edu

Environment and Reproduction

Component	Version
Python	3.13.5, in a virtual environment
PyCryptodome	3.23.0

Reproducing

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pycryptodome
```

Task 1. `diffie_hellman.py` runs the key exchange process using discrete logarithms. It takes prime q and primitive root alpha, creates public/private pairs for Bob and Alice, then creates a shared symmetric key for them using SHA256.

```
python diffie_hellman.py
```

Task 2. `diffie_hellman.py` then implements a MITM attack to show how an adversary can decrypt messages from Alice and Bob if they are able to tamper with the numbers used in the protocol, such as alpha.

```
python diffie_hellman.py
```

Task 3. `rsa.py` implements "textbook" RSA then takes advantage of RSA's malleability. It takes a variable number of bits (up to 2048), creates the public and private pairs for Bob and Alice, then encrypts their message so they can communicate with each other. It then takes advantage of textbook RSA's weaknesses so a bad actor can decrypt Alice and Bob's messages, violate the integrity of the messages, and forge signatures.

```
python rsa.py
```

Task 1: Implement Diffie-Hellman Key Exchange

Implementation

For this task, I wrote 4 functions. One to create a shared symmetric key, one for encryption, one for decryption, and one for the key generation process.

Creating a shared symmetric key. I implemented the function `derive_key` that takes an int s from the key generation process, converts it to bytes, then passes it through SHA256 and takes the first 16 bytes of the digest.

```
def derive_key(s: int) -> bytes:
    """Create the key using integer from DH key generation process"""
    s_bytes = s.to_bytes((s.bit_length() + 7) // 8 or 1, "big")
    return SHA256.new(s_bytes).digest()[:16]
```

Encryption and Decryption. I implemented `encrypt` and `decrypt` that handle encryption and decryption using AES-CBC.

```
def encrypt(key: bytes, iv: bytes, msg: str) -> bytes:
    """Encrypts using AES-CBC"""
    return AES.new(key, AES.MODE_CBC, iv).encrypt(pad(msg.encode(),
AES.block_size))


def decrypt(key: bytes, iv: bytes, cipher_text: bytes) -> str:
    """Decrypts using AES-CBC"""
    return unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(cipher_text),
AES.block_size).decode()
```

Key Exchange Process. I implemented `key_exchange` that takes a prime q and a generator alpha and runs the full protocol between Alice and Bob. Each side picks a random private value X in Z_q and computes their public value Y = alpha^X mod q. They exchange public values, and each computes the shared secret s by raising the other's public value to their own private value mod q. Both pass s through `derive_key` to get the same symmetric key k, then use it to encrypt a message to the other with AES-CBC using a shared IV.

```
def key_exchange(q: int, alpha: int) -> None:
    """Run the key generation process"""
    x_a = random.randint(1, q - 2)
    x_b = random.randint(1, q - 2)

    y_a = pow(alpha, x_a, q)
    y_b = pow(alpha, x_b, q)

    s_a = pow(y_b, x_a, q)
    s_b = pow(y_a, x_b, q)

    k_a = derive_key(s_a)
    k_b = derive_key(s_b)
```

```
    if k_a == k_b:
        print(f"Key made from a: {k_a}")
        print(f"Key made from b: {k_b}")
        print("KEYS MATCH")
    else:
        print("KEYS DO NOT MATCH")

    iv = get_random_bytes(16)

    c0 = encrypt(k_a, iv, "Hi Bob!")
    c1 = encrypt(k_b, iv, "Hi Alice!")

    print(f"Bob's received message: {decrypt(k_b, iv, c0)}")
    print(f"Alice's received message: {decrypt(k_a, iv, c1)}")
```

Results

Question 1: For task 1, how hard would it be for an adversary to solve the Diffie Hellman Problem (DHP) given these parameters? What strategy might the adversary take?

It would not be hard for an adversary to solve the Diffie Hellman problem with the tiny parameters. An adversary could brute force discrete logs by trying every exponent up to 36 until alpha^x mod q equals the public key. Then they could generate the secret key using the other person's public key.

Question 2: For task 1, would the same strategy used for the tiny parameters work for the large values of q and alpha? Why or why not?

No, this would not work with the large values of q and alpha. Brute forcing all the possible exponents at that large of a number would be infeasible. The number of attempts would scale exponentially with the number of possible exponents, which is huge for large prime numbers.

Task 2: Implement MITM key fixing & negotiated groups

Implementation

For this task, I wrote one function `mitm_key_fixing` that covers both attacks. It runs the same protocol as Task 1, but with Mallory tampering with the values Alice and Bob use. Which attack runs depends on whether a tampered alpha is passed in.

Tampering with Public Values. When no alpha is given, Mallory intercepts Y_A and Y_B and forwards q to each side instead. Alice and Bob then compute s by raising q to their private values mod q. Since q^X mod q = 0 for any X, both compute s = 0. Mallory knows this without either private key, so she derives the same k and decrypts both c0 and c1.

Tampering with the Generator. When a tampered alpha is given, Mallory uses that alpha instead, and the protocol otherwise runs normally. A bad alpha makes the s easy to derive. With alpha = q, alpha mod q is 0, so every public value and every shared secret is 0. Mallory again knows s = 0, derives k, and recovers m0 and m1.

```
def mitm_key_fixing(q: int, alpha=None) -> None:
    """Key exchange with Mallory replacing values with q or tampering with
alpha"""
    x_a = random.randint(1, q - 2)
    x_b = random.randint(1, q - 2)

    if alpha:
        y_a = pow(alpha, x_a, q)
        y_b = pow(alpha, x_b, q)
        s_a = pow(y_b, x_a, q)
        s_b = pow(y_a, x_b, q)
    else:
        s_a = pow(q, x_a, q)
        s_b = pow(q, x_b, q)

    k_a = derive_key(s_a)
    k_b = derive_key(s_b)

    iv = get_random_bytes(16)

    c0 = encrypt(k_a, iv, "Hi Bob!")
    c1 = encrypt(k_b, iv, "Hi Alice!")

    s_mallory = 0
    k_mallory = derive_key(s_mallory)

    print(f"Alice's s: {s_a}")
    print(f"Bob's s: {s_b}")
    print(f"Mallory's s: {s_mallory}")
    print(f"Mallory decrypting c0: {decrypt(k_mallory, iv, c0)}")
    print(f"Mallory decrypting c1: {decrypt(k_mallory, iv, c1)}")
```

Results

Question 3: For task 2, why were these attacks possible? What is necessary to prevent it?

Diffie-Hellman does not provide authentication in its protocol. So when Alice and Bob exchange values, they have no idea if these values are actually coming from each other. In order to fix this, they need to add authentication to know the values are coming from Alice and Bob. They can do this by signing the values such as alpha, q, and their public keys, so Mallory can't perform the MITM attack.

Task 3: Implement “textbook” RSA & MITM Key Fixing via Malleability

Implementation

For this task, I wrote functions for RSA key generation, encryption, and decryption, along with the same key derivation and AES-CBC helpers from Task 1. I then wrote three functions that exploit textbook RSA's malleability. One where Mallory recovers the shared key, one where Mallory alters an encrypted transaction, and one where Mallory forges a signature.

Key Generation. I implemented `key_generation` that takes a number of bits and returns the public key PU = (e, n) and private key PR = (d, n), with e = 65537. I use PyCryptodome's `getPrime` to generate primes p and q, then compute n = p * q and phi(n) = (p - 1)(q - 1). Since e is prime, it is coprime to phi(n) unless it divides it, so new primes are drawn if p = q or phi(n) is divisible by e. d is then computed as the multiplicative inverse of e mod phi(n) using `pow(E, -1, phi)`.

```python
def key_generation(bits: int) -> tuple[tuple[int, int], tuple[int, int]]:
    """Return PU = {e, n} and PR = {d, n}"""
    while True:
        p = getPrime(bits)
        q = getPrime(bits)

        phi = (p - 1) * (q - 1)

        if p != q and phi % E != 0:
            break

    n = p * q
    d = pow(E, -1, phi)
    return (E, n), (d, n)
```

Encryption and Decryption. I implemented `encrypt` and `decrypt`, which compute C = M^e mod n and M = C^d mod n. Encryption checks that the plaintext is less than n, since RSA can only operate on integers in Z_n. Messages are converted from ASCII to hex, then from hex to an integer before encrypting.

```python
def encrypt(public_key: tuple[int, int], m: int) -> int:
    """Create a cipher text from a given public key for RSA"""
    e, n = public_key

    if m >= n:
        raise ValueError("Plaintext must be smaller than n")
    return pow(m, e, n)


def decrypt(private_key: tuple[int, int], c: int) -> int:
    """Get back the plaintext using a private key for RSA"""
    d, n = private_key
    return pow(c, d, n)
```

Key Derivation and AES. I reused `derive_key` from Task 1, along with the AES-CBC functions renamed to `aes_encrypt` and `aes_decrypt`.

```python
def derive_key(s: int) -> bytes:
    """Create the key using integer from RSA key generation process"""
    s_bytes = s.to_bytes((s.bit_length() + 7) // 8 or 1, "big")
    return SHA256.new(s_bytes).digest()[:16]
```

```
def aes_encrypt(key: bytes, iv: bytes, msg: str) -> bytes:
    """Encrypts using AES-CBC"""
    return AES.new(key, AES.MODE_CBC, iv).encrypt(pad(msg.encode(),
AES.block_size))


def aes_decrypt(key: bytes, iv: bytes, cipher_text: bytes) -> str:
    """Decrypts using AES-CBC"""
    return unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(cipher_text),
AES.block_size).decode()
```

Recovering the Shared Key. I implemented `mitm_malleability` that follows the protocol where Bob sends Alice an encrypted value s. In place of Bob's ciphertext, Mallory sends c' = r^e mod n, where r is a value she picked. Encrypting only requires Alice's public key, so Mallory can do this without the private key. Alice decrypts c' and gets s = r, which Mallory already knows, so Mallory derives the same k and decrypts c0.

```
def mitm_malleability(bits: int) -> None:
    """Mallory replaces Bob's encrypted s with an encryption of a value she
chose"""
    public, private = key_generation(bits)
    _, n = public

    r = random.randint(2, n - 1)
    c_prime = encrypt(public, r)

    s_alice = decrypt(private, c_prime)
    k_alice = derive_key(s_alice)

    iv = get_random_bytes(16)
    c0 = aes_encrypt(k_alice, iv, "Hi Bob!")

    k_mallory = derive_key(r)
    print(f"Alice's s equals Mallory's r: {s_alice == r}")
    print(f"Mallory decrypts c0: {aes_decrypt(k_mallory, iv, c0)}")
```

Tampering with a Transaction. I implemented `transaction_tampering` as another example of how malleability can violate integrity. A transfer amount of $500 is encrypted with Alice's public key, and Mallory multiplies the ciphertext by 2^e mod n. Since Enc(a) * Enc(b) = (a * b)^e mod n, the receiver decrypts it to $1000. Mallory never learns the amount or the private key, and the receiver cannot tell the ciphertext was altered.

```
def transaction_tampering(bits: int) -> None:
    """Another example of how we can exploit RSA's malleability"""
    public, private = key_generation(bits)
    e, n = public

    amount = 500
```

```
    c = encrypt(public, amount)

    r = 2
    c_prime = c * pow(r, e, n)

    received = decrypt(private, c_prime)
    print(f"Sent: ${amount}, Received: ${received}")
```

Forging a Signature. I implemented `signature_forgery` that signs two messages m1 and m2 with Sign(m, d) = m^d mod n. Since m1^d * m2^d = (m1 * m2)^d mod n, Mallory multiplies the two signatures to get a valid signature for m3 = m1 * m2 without knowing d. Verifying the forged signature with the public key confirms it is valid.

```
def signature_forgery(bits: int) -> None:
    """Combine two valid signatures into a third valid signature"""
    public, private = key_generation(bits)
    e, n = public
    d, _ = private

    m1 = int("Hi".encode().hex(), 16)
    m2 = int("Bob".encode().hex(), 16)
    sig1 = pow(m1, d, n)
    sig2 = pow(m2, d, n)

    m3 = m1 * m2
    sig3 = sig1 * sig2

    print(f"Forged signature verifies: {pow(sig3, e, n) == m3}")
```

Results

Question 4: For task 3 part 1, while it's very common for many people to use the same value for e in their key (common values are 3, 7, 216+1), it is very bad if two people use the same RSA modulus n. Briefly describe why this is, and what the ramifications are.

Sharing e is fine because e is supposed to be public, and each person's key is still unique since they have their own p and q, which gives them a different d = e^-1 mod phi(n). If two people share n, they also share p and q, and if they use the same value for e, they will create the same d.

This means both users end up with identical key pairs, which has several ramifications. First, there is no confidentiality between them. Any message encrypted for Alice can be decrypted by Bob using his own private key and vice versa. Second, signatures no longer prove who sent a message. Since both users sign with the same d, a signature from Alice is the same as a signature from Bob, so either can forge the other's signatures.

Full Source Code

diffie_hellman.py

```
"""Diffie-Hellman Key Exchange Implementation"""

from Crypto.Cipher import AES
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes, random
from Crypto.Util.Padding import pad, unpad


Q = int(
    """
    B10B8F96 A080E01D DE92DE5E AE5D54EC 52C99FBC FB06A3C6
    9A6A9DCA 52D23B61 6073E286 75A23D18 9838EF1E 2EE652C0
    13ECB4AE A9061123 24975C3C D49B83BF ACCBDD7D 90C4BD70
    98488E9C 219A7372 4EFFD6FA E5644738 FAA31A4F F55BCCC0
    A151AF5F 0DC8B4BD 45BF37DF 365C1A65 E68CFDA7 6D4DA708
    DF1FB2BC 2E4A4371
    """.replace(" ", "").replace("\n", ""),
    16
)

ALPHA = int(
    """
    A4D1CBD5 C3FD3412 6765A442 EFB99905 F8104DD2 58AC507F
    D6406CFF 14266D31 266FEA1E 5C41564B 777E690F 5504F213
    160217B4 B01B886A 5E91547F 9E2749F4 D7FBD7D3 B9A92EE1
    909D0D22 63F80A76 A6A24C08 7A091F53 1DBF0A01 69B6A28A
    D662A4D1 8E73AFA3 2D779D59 18D08BC8 858F4DCE F97C2A24
    855E6EEB 22B3B2E5
    """.replace(" ", "").replace("\n", ""),
    16
)


def derive_key(s: int) -> bytes:
    """Create the key using integer from DH key generation process"""
    s_bytes = s.to_bytes((s.bit_length() + 7) // 8 or 1, "big")
    return SHA256.new(s_bytes).digest()[:16]


def encrypt(key: bytes, iv: bytes, msg: str) -> bytes:
    """Encrypts using AES-CBC"""
    return AES.new(key, AES.MODE_CBC, iv).encrypt(pad(msg.encode(),
AES.block_size))


def decrypt(key: bytes, iv: bytes, cipher_text: bytes) -> str:
    """Decrypts using AES-CBC"""
    return unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(cipher_text),
AES.block_size).decode()


def key_exchange(q: int, alpha: int) -> None:
```

```
    """Run the key generation process"""
    x_a = random.randint(1, q - 2)
    x_b = random.randint(1, q - 2)

    y_a = pow(alpha, x_a, q)
    y_b = pow(alpha, x_b, q)

    s_a = pow(y_b, x_a, q)
    s_b = pow(y_a, x_b, q)

    k_a = derive_key(s_a)
    k_b = derive_key(s_b)

    if k_a == k_b:
        print(f"Key made from a: {k_a}")
        print(f"Key made from b: {k_b}")
        print("KEYS MATCH")
    else:
        print("KEYS DO NOT MATCH")

    iv = get_random_bytes(16)

    c0 = encrypt(k_a, iv, "Hi Bob!")
    c1 = encrypt(k_b, iv, "Hi Alice!")

    print(f"Bob's received message: {decrypt(k_b, iv, c0)}")
    print(f"Alice's received message: {decrypt(k_a, iv, c1)}")


def mitm_key_fixing(q: int, alpha=None) -> None:
    """Key exchange with Mallory replacing values with q or tampering with
alpha"""
    x_a = random.randint(1, q - 2)
    x_b = random.randint(1, q - 2)

    if alpha:
        y_a = pow(alpha, x_a, q)
        y_b = pow(alpha, x_b, q)
        s_a = pow(y_b, x_a, q)
        s_b = pow(y_a, x_b, q)
    else:
        s_a = pow(q, x_a, q)
        s_b = pow(q, x_b, q)

    k_a = derive_key(s_a)
    k_b = derive_key(s_b)

    iv = get_random_bytes(16)

    c0 = encrypt(k_a, iv, "Hi Bob!")
    c1 = encrypt(k_b, iv, "Hi Alice!")

    s_mallory = 0
    k_mallory = derive_key(s_mallory)
```

```
    print(f"Alice's s: {s_a}")
    print(f"Bob's s: {s_b}")
    print(f"Mallory's s: {s_mallory}")
    print(f"Mallory decrypting c0: {decrypt(k_mallory, iv, c0)}")
    print(f"Mallory decrypting c1: {decrypt(k_mallory, iv, c1)}")


if __name__ == "__main__":
    key_exchange(37, 5)
    key_exchange(Q, ALPHA)
    mitm_key_fixing(Q)
    mitm_key_fixing(Q, Q)
```

rsa.py

```
"""RSA key generation and encryption/decryption implementation"""


from Crypto.Util.number import getPrime
from Crypto.Cipher import AES
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes, random
from Crypto.Util.Padding import pad, unpad


E = 65537


def key_generation(bits: int) -> tuple[tuple[int, int], tuple[int, int]]:
    """Return PU = {e, n} and PR = {d, n}"""
    while True:
        p = getPrime(bits)
        q = getPrime(bits)

        phi = (p - 1) * (q - 1)

        if p != q and phi % E != 0:
            break

    n = p * q
    d = pow(E, -1, phi)
    return (E, n), (d, n)


def encrypt(public_key: tuple[int, int], m: int) -> int:
    """Create a cipher text from a given public key for RSA"""
    e, n = public_key

    if m >= n:
```

```
        raise ValueError("Plaintext must be smaller than n")
    return pow(m, e, n)


def decrypt(private_key: tuple[int, int], c: int) -> int:
    """Get back the plaintext using a private key for RSA"""
    d, n = private_key
    return pow(c, d, n)


def derive_key(s: int) -> bytes:
    """Create the key using integer from RSA key generation process"""
    s_bytes = s.to_bytes((s.bit_length() + 7) // 8 or 1, "big")
    return SHA256.new(s_bytes).digest()[:16]


def aes_encrypt(key: bytes, iv: bytes, msg: str) -> bytes:
    """Encrypts using AES-CBC"""
    return AES.new(key, AES.MODE_CBC, iv).encrypt(pad(msg.encode(),
AES.block_size))


def aes_decrypt(key: bytes, iv: bytes, cipher_text: bytes) -> str:
    """Decrypts using AES-CBC"""
    return unpad(AES.new(key, AES.MODE_CBC, iv).decrypt(cipher_text),
AES.block_size).decode()


def mitm_malleability(bits: int) -> None:
    """Mallory replaces Bob's encrypted s with an encryption of a value she
chose"""
    public, private = key_generation(bits)
    _, n = public

    r = random.randint(2, n - 1)
    c_prime = encrypt(public, r)

    s_alice = decrypt(private, c_prime)
    k_alice = derive_key(s_alice)

    iv = get_random_bytes(16)
    c0 = aes_encrypt(k_alice, iv, "Hi Bob!")

    k_mallory = derive_key(r)
    print(f"Alice's s equals Mallory's r: {s_alice == r}")
    print(f"Mallory decrypts c0: {aes_decrypt(k_mallory, iv, c0)}")


def transaction_tampering(bits: int) -> None:
    """Another example of how we can exploit RSA's malleability"""
    public, private = key_generation(bits)
    e, n = public

    amount = 500
```

```
    c = encrypt(public, amount)

    r = 2
    c_prime = c * pow(r, e, n)

    received = decrypt(private, c_prime)
    print(f"Sent: ${amount}, Received: ${received}")


def signature_forgery(bits: int) -> None:
    """Combine two valid signatures into a third valid signature"""
    public, private = key_generation(bits)
    e, n = public
    d, _ = private

    m1 = int("Hi".encode().hex(), 16)
    m2 = int("Bob".encode().hex(), 16)
    sig1 = pow(m1, d, n)
    sig2 = pow(m2, d, n)

    m3 = m1 * m2
    sig3 = sig1 * sig2

    print(f"Forged signature verifies: {pow(sig3, e, n) == m3}")


if __name__ == "__main__":
    pu, pr = key_generation(2048)
    MSG = "Hi Bob!"
    temp = encrypt(pu, int(MSG.encode().hex(), 16))
    recovered = decrypt(pr, temp)
    PLAIN_TEXT = recovered.to_bytes((recovered.bit_length() + 7) // 8 or 1, "big")
    print(f"{MSG} -> {PLAIN_TEXT}")
    mitm_malleability(2048)
    transaction_tampering(2048)
    signature_forgery(2048)
```